An Idiomatic Formally Verified DER/X.690 Encoding Core in Rust (with Structural X.509 Parsing): A Dual-Layer Proof Architecture and an Honest Proof Envelope

Ivo Matijašević
Independent researcher
ivomatijasevic@gmail.com `https://github.com/ivmat/rs-verified-der`

Abstract

The DER encoding layer beneath X.509 is a recurring source of memory-safety bugs and, more insidiously, *parser differentials*—disagreements between implementations about whether the same bytes are a valid certificate. Unlike the higher X.509 profile, this layer has a small, checkable contract: accept a byte string *if and only if* it is the unique canonical DER encoding, and never panic doing so. We report on `der-verified`, an idiomatic Rust DER/X.690 encoding core built so that its correctness evidence *re-runs from a fresh clone*. The crate combines two verification lineages over the same source: a mandatory, exhaustive-but-bounded Kani/CBMC floor of 161 proof harnesses across 25 modules—each harness proving its own asserted property (safety, and per codec some of round-trip, minimality, or rejection) over a stated bound—and an optional, additive set of three Aeneas→Lean 4 "lids" that lift three codecs (where an ∀-length guarantee is most valuable and the functional model is cleanest) to inputs of *any* length, gated by a drift-detecting re-extraction check. We describe (i) the dual-layer architecture and the build engineering that lets a stable-toolchain Kani gate coexist with a nightly-pinned extraction pipeline without duplicating source; (ii) two API- and proof-structuring techniques that make an idiomatic decoder BMC-tractable—a symbolic-length discharge rule for modular stub proofs, and a two-tier strict/permissive decode API that closes the trailing-byte differential surface at the crate's top-level entry points; (iii) a measured cost profile in which CBMC's symbolic-execution *formula construction*, not SAT solving, was the wall on our hardest nested-container proof, and solver choice a modest, harness-specific lever; and (iv) an "honest proof envelope" discipline—a first-class manifest of bounds, assumptions, stubs, and explicit non-goals—that we position as a concrete, author-side answer to recent critiques of "verification theatre." We also document a silent Aeneas failure mode (extraction that "succeeds" by degrading a function to a bodyless axiom) and its refactor fix. None of the individual ingredients is new; the contribution is the specific, reproducible combination on a real DER/X.509 target, reported honestly.

1 Introduction

A large fraction of TLS handshakes, code signatures, and PKI checks rest on parsing X.509 certificates, which are serialized in DER—the Distinguished Encoding Rules, a canonical subset of ASN.1's Basic Encoding Rules (X.690). That encoding layer has a long, ugly track record: memory-safety bugs in C parsers, and *parser differentials*, where two implementations disagree on whether a given byte string is a valid certificate. Differential-testing work such as *Frankencerts* [4] demonstrated how routinely production stacks diverge—one library rejects bytes another accepts, one normalizes a non-canonical length another treats as a distinct value—and such disagreements are a recurring source of certificate-confusion and validation-bypass bugs. Empirical studies of in-the-wild X.509 parsing [5] continue to find such divergence.

DER is, in principle, the friendly case. Every value has *exactly one* valid encoding, so a decoder has a crisp, checkable contract: accept a byte string if and

only if it is the unique canonical DER encoding, and never panic. That contract is small enough to *prove*, and proving it is a real, if narrow, reduction of attack surface.

This paper is an experience report on `der-verified` [1], a from-scratch Rust DER/X.690 encoding core (published to crates.io) in which every public codec carries machine-checkable evidence, and that evidence re-runs from a fresh clone—the proofs are the product, not a badge. The crate is `#![forbid(unsafe_code)]`, dependency-free, and allocation-free on its decode paths. It is deliberately *idiomatic* Rust rather than a verification-DSL artifact: the same source that ships is the source that is verified.

What is and is not new. We claim no new verification *technique*. Kani/CBMC bounded model checking [8], Aeneas/Charon functional translation of Rust to Lean 4 [9], and—as established practice in high-assurance verification—layered assurance, explicit TCB/scope reporting, and deliberate spec-scoping are all prior art. The nearest neighbour in the same domain, Verdict [2], is a single-layer, Verus-verified end-to-end X.509 *validator*. Our contribution is instead an *artifact*: a specific, reproducible combination—an idiomatic Kani-verified Rust DER crate with optional extraction-based unbounded lids, a drift gate tying the two together, and an explicit "what is *not* proven" discipline—reported with its costs and its gotchas rather than only its successes. We believe the honest combination is itself worth the record; recent work on "verification theatre" [7] argues, with thirteen concrete escapes from formally verified crypto libraries, that the gap between what a project's marketing claims and what its proofs actually cover is a live, systemic risk. The proof envelope discipline in §5.2 is designed to make that gap small and legible from the author's side.

Scope of the verified core. In scope (verified): the DER encoding layer—identifier (tag) and definite-length fields, and the canonical content codecs `BOOLEAN`, `INTEGER` (both `i64` and arbitrary-magnitude), `NULL`, `OBJECT IDENTIFIER`, `BIT STRING`, `OCTET STRING`, `ENUMERATED`, the ASCII-restricted strings, `UTF8String`, `UTCTime`, `GeneralizedTime`, `SEQUENCE`, and `SET OF` member ordering (§11.6). The `x509_*` modules parse RFC 5280 objects (`AlgorithmIdentifier`, `SubjectPublicKeyInfo`, `Name`, `Validity`, `Extension`, `TBSCertificate`, `Certificate`) by composing the verified codecs, but interpret *no* algorithm, key, signature, or trust semantics. Out of scope (and explicitly *not* proven): cryptography, path/trust validation, and the full RFC 5280 profile. This fence is not incidental—it is the crate's central honesty claim (§5,§5.2). We stress up front that the claim is *per-codec and per-bound*: which property (round-trip, minimality, an acceptance biconditional, or panic-freedom alone) is proven, and under what bound, varies by module and is listed in the manifest; the `x509_*` modules carry structural panic-freedom evidence, not verified RFC 5280 semantics; and "if and only if" holds only where a named harness or Lean theorem proves both directions for the implemented profile.

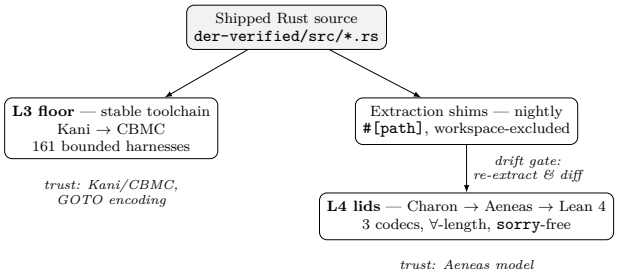

Figure 1: Two lineages over one source. The L3 stable Kani floor is mandatory; the L4 nightly extraction lids are optional and additive, tied to the same bytes by the drift gate. Each layer's trust boundary is labelled; the two are complementary, not independent (both begin with the same Rust).

2 The dual-layer architecture

The verification stack has two lineages over the *same* Rust source (Figure 1). We name them by assurance depth.

L3 — the floor (mandatory, bounded). 161 Kani proof harnesses across 25 modules. Kani compiles each `#[kani::proof]` to CBMC and discharges it as a bit-precise SAT/SMT query. Every harness proves, by Kani's default checks, memory safety, absence of panics, and absence of arithmetic overflow on its input domain, plus the functional properties each harness asserts: decode/encode round-trip, canonicality/minimality, and rejection of malformed or non-canonical encodings. "Bounded" has a precise meaning here: each harness constructs a fixed-size symbolic input (`kani::any()` byte arrays of a chosen length), optionally narrowed by `kani::assume(...)` preconditions, and unrolls loops to a stated `#[kani::unwind(N)]` depth (depths in use range 1–22, most codecs at 16). The proof is therefore *complete over that bounded input domain and no larger*. Running the floor requires only `cargo` and the Kani toolchain.

L4 — the lids (optional, additive, unbounded). Bounded proofs leave a nagging question: what about inputs larger than the bound? For three codecs—`length` (§8.1.3), `big_integer` (§8.3), and `oid` (§8.19)—the Rust is additionally translated, Rust → Charon → Aeneas → Lean 4, into a pure functional model, and the corresponding properties are proven in Lean for inputs of *any* length, `sorry`-free (Table 1). The L4 layer lifts exactly the codecs where an ∀-length guarantee is most valuable and the functional model is cleanest.

The property that ties the layers together. Two design choices make the two-layer claim sound rather than cosmetic. First, L4 is *guarded*: the driver script `lean/check_lean.sh` no-ops (exit 0) when the Aeneas/Lean toolchain is absent, so the full gate still passes on the Kani floor alone on a machine without the heavy extraction stack. The lids are additive assurance, never a build prerequisite—a practical necessity if contributors and CI are not all to be forced to provision a proof assistant. Second, and more important, each lid *re-extracts its model from the exact shipped `.rs` on every run* and the gate *fails closed on drift* between the regenerated extraction and the committed Lean source. This closes the classic extraction-verification trap in which the proved model silently diverges from the shipped code: the gate mechanically ensures the committed Lean model is *syntactically identical* to what the pinned Aeneas/Charon pipeline regenerates from the *current* shipped `.rs` (the same source the L3 floor proves), not a stale hand-transcribed snapshot. The *semantic* correspondence between that model and the shipped Rust remains trusted (the trust base, below); the gate closes the source-drift hole, not the translation gap (and, per §6, not a silent axiom-degradation hole). The lid additionally fails closed on any `sorry` (checked mechanically for `sorryAx` / "declaration uses `sorry`").

Codec	Unbounded property proven (∀-length)
`length`	every branch of `decode_length`; round-trip canonicality
`big_integer`	minimality biconditional (validate side); encode round-trip
`oid`	OID canonical-form biconditional (validate side)

Table 1: The three L4 Aeneas→Lean 4 lids. All `sorry`-free; the full axiom set each rests on—beyond Lean's standard `propext` / `Classical.choice` / `Quot.sound`, the named Aeneas-Std spec axioms and `bv_decide`'s certificate axiom—is disclosed via `#print axioms` (`sorry`-free is not axiom-free).

Trust base (both layers). Neither layer is trust-free, and we enumerate both boundaries rather than hide them. *L3* trusts Kani's Rust→GOTO encoding, CBMC and its underlying SAT/SMT solver (the proofs are not independently certificate-checked), and the fact that Kani's bundled toolchain is a different compiler instance than the `stable rustc` that ships the crate. *L4* additionally trusts the Lean 4 kernel, the disclosed non-standard axioms (the Aeneas-Std spec axioms and `bv_decide`'s certificate axiom), and the *Aeneas model* of the Rust—the Charon/Aeneas translation is not itself formally verified against `rustc` semantics (the standard Aeneas assurance boundary). Both layers also assume the extract shims' features/`cfg`s match the shipping build.

The two layers are therefore *complementary*, not semantically independent cross-validation: both begin with the same Rust and rustc-derived machinery.

Reconciling two toolchains without copying source. A build-engineering tension threatens the single-source-of-truth property. The main Kani gate must stay pinned to the *stable* Rust channel; Charon (the Rust-to-LLBC front-end driving Aeneas) requires its own pinned *nightly* (`nightly-2026-06-01`). Copying the extracted source would risk exactly the drift the L4 gate exists to prevent; repointing the whole crate to nightly would risk the Kani gate. We resolve this with one small *shim crate per extracted module* (`lean/extract/`, `lean/extract-bigint/`, `lean/extract-oid/`), each of which (1) `#[path]`-includes the *exact same* source file from `der-verified/src/` rather than duplicating it, (2) carries its own `rust-toolchain.toml` pinned to Charon's nightly, and (3) is *excluded* from the root Cargo workspace, so it never participates in the main `cargo build`/`test`/`kani` invocations and cannot regress the stable Kani gate. The individual Cargo mechanisms (per-directory toolchain pins, workspace `exclude`, `#[path]` inclusion) are standard; combining them into a per-module shim that isolates a nightly extraction step from a stable proof gate on one source file is the reusable idiom. Together with the drift gate, the source-of-truth link is closed: the same source bytes proven by Kani are the bytes extracted and proven by Lean (the Aeneas translation step over those bytes remains trusted, as above).

3 Making an idiomatic decoder BMC-tractable

Two techniques were necessary to verify idiomatic (non-DSL) Rust with a bounded model checker at this scale. Both are stated as general rules; the DER setting is the instance.

3.1 Symbolic-length discharge for modular stub proofs

Large compositions are proven *modularly*: an already-verified inner parser is replaced in an outer harness by a `#[kani::stub]` carrying the inner function's proven postcondition, so CBMC reasons only about the composition glue. The crate uses four such stubs across three harnesses, forming a DAG (`x509_certificate` → `x509_tbs_certificate` → {`x509_name` → its RDN lemma, `x509_extension`}); each link is a real function separately proven panic-free.

The subtle part is *how* the stub's contract is discharged. A DER parser's control flow is length-dependent: every length check reads `input.len()`, and the composition invokes a stubbed sub-parser on *suffix slices* of many different lengths, not only the outer buffer's full length. Discharging the stub's postcondition lemma only at one fixed length therefore leaves the compositional argument *unsound*: every shorter call length relies on an undischarged contract instance, while the harness still passes. The rule we adopted repo-wide is to discharge each stub's contract over a *symbolic* input length:

```
let len = kani::any();
kani::assume(len <= buf.len());
// prove the postcondition of f(&buf[..len])
```

Symbolic length closes the previously omitted *length-domain* obligation, but it is only one part of a sound modular replacement. We state the rule for the case that actually holds in this crate—the stubbed callees are *pure functions over an immutable input slice* whose entire observable effect is their return value (an `Ok(used)` byte count or a rejection), no mutation, no aliasing, no side channel. Under that restriction, and checked by hand per stub (see also §8), the replacement is sound when: (a) the real callee is separately proven to satisfy its *exact relational contract*—not merely panic-freedom, but the full result postcondition the stub asserts (e.g. `validate_rdn`'s $2 \leq used \leq input.len()$), including how many input bytes it consumes; (b) *bound dominance*—the discharge harness's buffer bound covers every suffix length the outer call sites can pass ($N_{lemma} \geq N_{caller}$); a symbolic discharge over a too-small buffer has the same shape and the same silent "both versions verify" fail-

ure; (c) every `kani::assume` precondition in the discharge harness is established at each outer call site; and (d) the stub is a genuinely over-approximating nondeterministic stand-in (a `kani::any()` result constrained to that contract), so its outcomes—value *and* consumed-byte count—are a superset of the real callee's. The per-edge mapping (which harness stubs which sub-parser, and where each stubbed callee's own contract proof lives) is the manifest's stub table; a fully mechanized linkage would use Kani function contracts (below) so (a) is enforced rather than argued. The move from the old fixed-length form to the symbolic one is a strict *generalization* (it widens the covered domain from the single length N to $0..=N$ and subsumes the old proof), and was empirically nearly free because the discharge buffers were already fully symbolic; it made the crate's "up to N octets" claims actually true.

Kani's function-contracts feature (`-Z function-contracts`, `#[kani::requires]`/`#[kani::ensures]` with `proof_for_contract` / `stub_verified`) is the institutional mechanism for exactly this "discharge a contract, then substitute it" pattern; we used hand-written `-Z stubbing` stubs instead, but the same fixed-vs-symbolic-length discharge gap can arise in a `proof_for_contract` harness, so the bound-dominance rule is complementary to that feature rather than a gap in it. We have not found the specific failure mode called out explicitly in the mainstream Kani/CBMC documentation [8]; it is easy to miss precisely because both the fixed- and symbolic-length versions verify.

3.2 A two-tier decode API closes the trailing-byte differential

X.690 §8.1.1.1 fixes the shape of a DER encoding (identifier, length, contents octets)—so a byte string that *purports to be one complete value* is well-formed only if it is exactly one such TLV with nothing after it. (This is a property of the complete-object API contract, not a blanket rule that any decoder input may contain only one TLV: recursive ASN.1 encodings are precisely sequences of component encodings.) A recursive descent parser therefore legitimately needs to decode "one TLV, then whatever comes after" at every *inner* level—an inner value naturally has a byte suffix belonging to the next sibling. Collapsing these two needs into one function with an ambiguous contract is exactly how trailing-byte parser differentials arise. We split the API into two statically distinct tiers:

- `decode_tlv` / `decode_sequence_tlv`: non-strict, consume one TLV and *permit* a trailing suffix; used *only* to drive recursive parsing of inner values.
- `decode_tlv_strict` / `decode_sequence_tlv_strict`: strict, require the input to be exactly one TLV and return a distinct `TrailingData` error otherwise; used as the *only* top-level entry points (`parse_certificate` uses the strict form, so appended bytes are rejected at the outer `SEQUENCE`).

Each tier's boundary property is separately machine-checked, on a *bounded* domain: one harness (`decode_tlv_structure`) proves over a symbolic buffer that an accepted TLV consumes exactly *header* + *declared_length* bytes and never over-reads; another (`strict_rejects_trailing`) proves the strict wrapper returns `TrailingData` on a valid TLV followed by *one* arbitrary trailing byte. Rejection of longer trailing suffixes follows from the exact-consumption harness plus the strict wrapper's remainder check, but that is a source-level argument, not the harness statement. Two honest limits: the surface is closed *at the crate's top-level entry points* (`parse_certificate` uses the strict form), and the permissive `decode_tlv` / `decode_sequence_tlv` remain `pub`—so a downstream caller could still use the non-strict tier at top level. The discipline is enforced by naming and convention for external users, not by Rust's type or visibility system; making the permissive tier non-public (or exposing a typed recursive-vs-complete distinction) would harden it further. The underlying DER rule is old and the strict/lenient distinction is parser folklore; packaging it as two independently *proven* API tiers is the transferable idiom, applicable to any recursive TLV-shaped format with the same "inner permissive, outer strict" structure. It is philosophically close to EverParse/LowParse's per-combinator non-

malleability and exact-consumption proofs [3], but organized around a Rust API split rather than combinator composition.

4 What actually costs

We measured the full Kani suite and report it honestly, because "how expensive is this to reproduce?" is a first-order question for an artifact whose whole pitch is re-runnability. Numbers below are point-in-time on the stated hardware; we treat the *tiers* as durable and the exact seconds as indicative.

The wall is formula construction, not solving. The hardest harness proves that validating an X.509 `Name`—a `SEQUENCE OF SET OF AttributeTypeAndValue`, doubly nested and variable-count, with DER-mandated canonical `SET OF` ordering—never panics. Proving this over fully symbolic bytes made CBMC re-derive the pairwise padded-byte ordering comparison (`set_of::cmp_padded`) across every partition of the input, inside a three-deep loop nest. Memory climbed past **100 GB during symbolic execution**, *before the SAT solver ran.* Crucially, reducing the unwind depth and shrinking the input buffer—both tested independently—*barely moved it.* The dominant driver was the loop-nest product of the ordering machinery, not the loop bound. We frame this as an artifact observation and a diagnostic procedure rather than a general law: for *this* harness, formula generation dominated, so on a nested-container BMC proof one should first measure which phase (formula construction versus solving) dominates, and whether the unwind/buffer knobs actually move it, *before* assuming they will. If they do not, the fix is proof restructuring, not more compute. The general phenomenon (nested-loop CBMC formula blowup dominating memory) is documented BMC behaviour [11]; the measured DER instance and the "knobs don't help here—it's structural" diagnosis are our contribution.

The fix is the standard modular move. Following §3.1, the heavy inner layer `validate_rdn` (one `RelativeDistinguishedName`) is proven panic-free once at its natural scale (∼17 GB), establishing a post-condition ($2 \leq \mathit{used} \leq \mathit{input.len}()$); the outer `validate_name` harness then stubs `validate_rdn` with that contract and reasons only about the `Name` glue, in ∼510 MB. Same theorem, split from an intractable >100 GB monolith into two pieces at commodity large-memory-workstation scale (the ∼17 GB piece still exceeds a 16 GB laptop and runs on the reference box)—and the reason `-Z stubbing` is required to reproduce the floor.

Per-module cost profile. Table 2 summarizes the tiers, measured on a deliberately modest 16 GB Apple-silicon laptop with `cargo-kani 0.67.0` / CBMC 6.8.0. Cost is heavily right-skewed: the large majority of the 161 harnesses finish sub-second to a few seconds; a moderate tier runs single-digit to tens of seconds; a heavy tail is dominated by the `set_of` §11.6 padded comparison re-derived over symbolic content; and two harnesses exceed a 16 GB laptop and are documented as reference-CI (large-box) checks. We stress that counts are inventory, not coverage, and that *every* listed harness verifies successfully—nothing in the table is a failure, only a cost.

Solver choice is a modest, harness-specific lever. On the slowest single harness (`set_of::tag_correctness`, ∼256 s on the default solver), we measured the CBMC back-ends: `cadical` at ∼220 s (∼14% faster) but higher peak memory (∼6.6 GB), and `kissat` which did *not* finish within a 5-minute budget despite lower memory (∼4.7 GB). The effect is modest and direction-inconsistent, matching the SAT-competition folklore that solver performance is instance-dependent with no universal winner. We therefore pin *no* solver crate-wide and recommend measuring the specific slow harness one is iterating on. Kani's supported backends are documented [8]; we simply supply one concrete comparative data point.

Reproducibility. CI runs the memory-tractable share of the floor (135 of 161 harnesses, sharded by module across three parallel runners) on every push;

Tier / representative harness	Time	Peak RSS
Fast majority (`length`, `oid`, `boolean`, ...)	< 1–few s	small
`restricted_string` (26-harn. total)	∼21 s	—
`x509_certificate` never-panics	∼50 s	—
`utf8_string::roundtrip`	∼109 s	—
`set_of::tag_correctness`	∼256 s	∼5 GB
`x509_name::validate_rdn`	—	∼17 GB†
`x509_extension` never-panics	DNF†‡	large

Table 2: Kani cost tiers, measured on a 16 GB Apple-silicon laptop (Kani 0.67.0 / CBMC 6.8.0). Rows are individual harnesses except where marked "total." †exceeds the 16 GB laptop's physical memory; runs on the 16-core / 29 GB Linux reference box (where the full floor is ∼40 min at ∼20 GB peak). ‡did not finish within a 5-minute per-harness budget on the laptop.

the 26 heaviest (`set_of`, `sequence`, `x509_extension`, `x509_certificate`, `x509_tbs_certificate`, `x509_name`) peak above a standard 7 GB hosted runner and are documented as local-milestone checks. On a 16-core / 29 GB Linux reference box the three CI shards run in ∼4–5 min wall (`codecs-a` 84 harnesses ∼28 s, `codecs-b` 42 ∼40 s, `utf8` 9 ∼247 s), and the full suite verifies end to end with zero failures.

5 Scoping honestly

5.1 Every narrowing is a recorded decision, not a silent gap

`der-verified` is a *strict, deliberately narrowed* profile of DER/X.509, not a best-effort partial implementation, and the difference is made explicit. Concrete narrowings—each an explicitly recorded design decision with its own rationale and status in a decisions ledger, and cross-referenced from the manifest's "deliberate deviations" and "what is NOT proven" sections—include: capping `INTEGER` at `i64` with a *separate* `big_integer` module for arbitrary magnitude (keeping each module's proof domain simple and independently boundable); rejecting BER's constructed/segmented `OCTET STRING` form (simultaneously a scope narrowing *and* a parser-differential hardening); validating only `SET OF` member ordering (§11.6) while leaving general `SET` (§10.3) out of scope; rejecting the leap-second `SS=60` time value as a profile choice, reducing time validation to single-field range checks rather than full calendar semantics. Deliberate spec-scoping for tractable verification is standard practice (seL4, CompCert, and—in this very domain—Verdict [2], which necessarily scopes to policy-specific validators). The discipline we emphasize is procedural: narrow at named points, record each narrowing as an individually documented decision with its rationale and status, and surface the resulting boundary as an explicit list rather than an implicit gap a user must discover the hard way. It is the difference between "we verified X" and "we verified exactly *this* profile of X, and here is precisely where the fence is and why."

5.2 The proof envelope as a first-class, gated artifact

The single most reusable output of the project is not any one proof but the discipline around the claim. `der-verified` publishes `PROOF_MANIFEST.md` as a dedicated, README-gated document—the mandatory first read—whose explicit purpose is to stop the crate's own impressive-sounding numbers from being over-read as completeness. Its opening line: *"Counts are inventory, not coverage ... describes how much verification exists, not a guarantee that every reachable behaviour is covered."* The manifest then makes the actual claim precise and auditable via:

- a **toolchain-pins table** (the recorded `rustc` version plus a floating `stable`-channel configuration—so a future clone may select a different stable compiler—and the exact Kani/CBMC/Lean/Aeneas/Charon versions/revisions the claims were checked against, with the Aeneas/Charon revisions mechanically *enforced* by the drift gate);

- a **per-module inventory table** (harness count, `#[kani::unwind]` range, and whether an L4 lid exists), so a reader sees which module has what depth of assurance rather than an aggregate count;
- explicit disclosure that **130 `kani::assume` preconditions** narrow the proved domain, with the framing that the properties hold *for inputs satisfying the assumptions* and inputs outside them are simply not claimed;
- a **modular-proof stub table** (which harness stubs which sub-parser, and where each stub's own proof lives), so compositional soundness is auditable rather than asserted;
- a dedicated **"what is NOT proven" scope fence** (no cryptography, no full profile semantics, bounded except the three L4 codecs, the Aeneas-translation trust gap), stated as prominently as what *is* proven.

Stating assumptions and TCB explicitly is best practice in formal methods; the reusable template here is a single crate-level manifest, cross-linked as the *primary trust artifact*, with the "counts are inventory" framing and a per-module bound/assumption/stub table. We position this deliberately against Kobeissi's "verification theatre" [7], which studies the same underlying *verification boundary* problem—the interface between machine-checked code and trusted-but-unverified code—and documents thirteen bugs escaping formal verification in production crypto libraries, from an *external audit* angle. The manifest is the complementary, *author-side* move: make the boundary a first-class, gated document by design, so the gap between claim and coverage is small and legible before an auditor has to find it. Calling `der-verified` "a verified X.509 parser" would be a lie; it is a verified *encoding* layer, and the manifest keeps that honest.

6 A toolchain gotcha worth reporting

Preparing idiomatic Rust for Aeneas extraction surfaced a silent, dangerous failure mode. Aeneas's Lean 4 backend does not (yet) support a `return` nested two loop-levels deep—the crate's original `validate_oid` had a `return Err(Truncated)` inside a `while` inside an outer loop. The documented Aeneas limitation is "returns inside nested loops ... are not supported yet" [9]; what is *not* documented is the *consequence*. Rather than erroring, extraction "succeeds" by silently degrading the function to a **bodyless `axiom`**. A bodyless axiom is unconstrained by the function's implementation, so it loses all correspondence to the Rust—any lid "proving" a property about it says nothing about the shipped code—yet the pipeline reports success. A contributor could ship a "verified" lid that is actually an unconstrained axiom without noticing, unless they specifically inspect the generated Lean for `axiom` declarations where a real definition was expected.

The fix was a source refactor: rewrite `validate_oid` as a *single loop* with an explicit `at_subid_start` state variable, so every early `return` sits at loop-depth 1. The refactor was *regression-checked* rather than assumed safe—the crate's five `oid` Kani harnesses and its 294 tests pass against it—which is bounded, example-based regression evidence, *not* a full behavioural-equivalence proof between the two implementations. The refactored `oid` harnesses are also inexpensive today ($\sim$0.04 s per the cost note). The general recipe (single loop + explicit state, keeping early returns at depth 1) is a reusable pattern for Rust being prepared for Aeneas extraction, since early returns inside nested loops are a common Rust parser idiom. We reported the silent-axiom behaviour upstream (Aeneas issue #1221 [10]); it is distinct both from the documented general limitation and from the closest pre-existing report, issue #1206, which is a *hard-error* failure mode (a `for` loop with conditional `break`) rather than silent degradation. Note that the crate's own L4 gate does *not* fully catch this: the `sorry` check does not fire on a bodyless axiom, and the drift gate only compares against the *committed* Lean, so a regenerate-and-commit workflow could launder a degraded axiom in. Closing it properly needs a mechanical *expected-definition* gate—fail unless each lid's target function is emitted as a body-bearing definition (equivalently, an allowlist for the extracted axioms/constants). Un-

til then we grep extracted Lean for unexpected `axiom` declarations as a defensive check; we flag this residual hole candidly rather than imply the gate already closes it.

7 Related work

Verified X.509 in the same domain. Verdict [2] is the closest published prior art: an end-to-end, Verus-verified Rust X.509 *validator* with customizable, first-order-logic validation policies and a proof-producing compiler, evaluated on over ten million real certificates. It is single-layer (deductive verification, no BMC floor and no extraction lids), targets the full validation problem (parsing, path building, path validation), and verifies the Rust source directly—so it has no extraction step and no drift surface to gate. Our target is narrower and lower—the encoding brick—but the artifact is a different *architecture*: an idiomatic bounded floor plus optional unbounded extraction lids over the same source. A recent Rust→Charon→Lean pipeline with AI-prover assistance [6], applied to zero-knowledge-proof code, shares our toolchain family but is a sequential pipeline, not a BMC-floor-plus-lid dual gate with drift detection.

Unverified Rust DER/X.509 crates. The incumbent Rust ecosystem—RustCrypto's `der` (which underpins `x509-cert`) and `rasn`—is widely used but not formally verified. We built a from-scratch idiomatic crate rather than retrofit proofs onto one of these because a Kani/Aeneas-friendly proof surface (bounded symbolic buffers, stubbable seams, single-loop functions amenable to extraction) is far easier to design in than to retrofit onto code written without verification in mind; the cost is reimplementing the encoding layer, and the benefit is that the shipped code and the verified code are identical by construction.

Parser differentials. Frankencerts [4] and subsequent in-the-wild studies of X.509 parsing [5] motivate the problem class: the trailing-byte surface that §3.2 addresses at the crate's top-level entry points, and the malformed/non-canonical-encoding class the per-codec proofs reject. EverParse/LowParse [3] provides verified parser combinators with non-malleability and exact-consumption guarantees—philosophically close to "prove each boundary property"—organized around combinator composition rather than an idiomatic Rust API.

Verification honesty. Kobeissi's "verification theatre" [7] is the most direct context for §5.2: it defines the verification boundary as the structural failure pattern and argues, from an external-audit stance, that overselling verification completeness is a live risk. The proof-envelope manifest is the author-side counterpart. Stating TCB and scope explicitly is long-established practice in high-assurance formal methods; our contribution is a concrete, reusable crate-level template with mechanical drift enforcement.

BMC cost. The nested-loop CBMC formula-construction blowup we measure (§4) is a documented characteristic of BMC tools [11, 12]; our contribution is a concrete, root-caused instance on a real DER/X.509 target and the diagnostic that unwind/buffer tuning does not move a structurally dominated cost.

8 Limitations and threats to validity

We collect the caveats stated throughout, since an experience paper's value depends on them being legible in one place.

Assurance scope. The L3 floor is *bounded*: properties hold over fixed-size symbolic buffers, under 130 disclosed `kani::assume` preconditions, up to stated unwind depths; inputs outside those domains are not claimed. Only three codecs have ∀-length L4 lids. The correctness of the harness *assertions* and the stub *contracts* is itself a manual modelling step; a wrong assertion proves the wrong thing. The modular proofs rest on the refinement obligation of §3.1 (bound dominance, precondition discharge, over-approximating stubs), checked by hand per stub, not

by a machine-checked meta-theorem.

Trust base. L3 trusts Kani/CBMC (and its SAT/SMT solver, not certificate- checked) and the Rust→GOTO encoding; L4 additionally trusts the Lean 4 kernel, the disclosed axioms, and the Charon/Aeneas translation against `rustc` semantics. Both layers begin with the same source and rustc-derived machinery, so they are complementary, not independent. `sorry`-free is not axiom-free: the L4 proofs rest on named Aeneas-Std spec axioms and `bv_decide`'s certificate axiom (disclosed via `#print axioms`). The extract shims' features/`cfg`s are assumed to match the shipping build. Moreover the L4 gate does *not* mechanically exclude extraction that degrades a function to a bodyless axiom (§6): the `sorry` check does not fire on it and the drift gate compares only against the committed Lean, so until an expected-definition gate exists this is guarded by a defensive grep, not a fail-closed check.

Reproducibility and measurement. The default gate conditionally *skips* L4 when the extraction toolchain is absent; a full-evidence run must provision it. The Rust side is a *floating* stable channel, so a future clone may compile with a different `rustc`. All performance numbers are point-in-time on the two stated machines and move with tool versions.

Behavioural claims. The OID refactor (§6) is regression-checked, not proven equivalent. The trailing-byte surface is closed for the crate's own top-level entry points; the permissive tier remains public. Regression tests, bounded proofs, and unbounded lids are three distinct grades of evidence, and the manifest—not any aggregate count—is the actual claim.

9 Conclusion

You cannot (yet) formally verify "X.509 is safe." But you can carve off the encoding layer—the part with a crisp canonical-form contract and a bad CVE history—and prove that brick does not panic and accepts only canonical bytes, with evidence anyone can re-run from a fresh clone. `der-verified` does this with a mandatory bounded Kani floor and optional unbounded Aeneas→Lean lids over the same idiomatic source, tied together by a drift-detecting gate, and reported through a first-class honest proof envelope. None of the ingredients is individually new; the reproducible combination, its measured costs, and its gotchas—reported honestly rather than only in their best light—are the artifact. It is also a template for doing the same to the layers above.

Availability. `der-verified` is on crates.io and at `https://github.com/ivmat/rs-verified-der`. From a fresh clone, `./check.sh` reproduces the L3 floor (161 Kani harnesses) and the 294 tests; it additionally re-extracts and checks the three Aeneas→Lean lids *when* the pinned extraction toolchain is present, and otherwise skips them and still passes on the floor (§2). A full-evidence reproduction (floor + lids + the honest proof envelope) therefore requires provisioning that toolchain, as the README documents.

References

[1] I. Matijašević. *der-verified: a formally verified DER/X.690 encoding core in Rust.* crates.io / `https://github.com/ivmat/rs-verified-der`, 2026.

[2] Z. Lin, M. McLoughlin, P. Singh, R. Brennan-Jones, P. Hitchcox, J. Gancher, and B. Parno. *Towards Practical, End-to-End Formally Verified X.509 Certificate Validators with Verdict.* In *34th USENIX Security Symposium*, 2025. `https://www.usenix.org/conference/usenixsecurity25/presentation/lin-zhengyao`.

[3] T. Ramananandro, A. Delignat-Lavaud, C. Fournet, N. Swamy, T. Chajed, N. Kobeissi, and J. Protzenko. *EverParse: Verified Secure Zero-Copy Parsers for Authenticated Message Formats.* In *28th USENIX Security Symposium*, 2019. (LowParse is the underlying combinator library.) `https://project-everest.github.io/everparse/`.

[4] C. Brubaker, S. Jana, B. Ray, S. Khurshid, and V. Shmatikov. *Using Frankencerts for Automated Adversarial Testing of Certificate Validation in SSL/TLS Implementations.* In *IEEE Symposium on Security and Privacy*, 2014.

[5] S. Tatschner, S. N. Peters, M. P. Heinl, T. Specht, and T. Newe. *ParsEval: Evaluation of Parsing Behavior using Real-world Out-in-the-wild X.509 Certificates.* In *Proc. 19th Int. Conf. on Availability, Reliability and*

Security (ARES), 2024. https://arxiv.org/abs/2405.18993.

[6] N. Klaus, J. Conejero, and P. Tolmach. *A Rust-to-Lean Verification Pipeline with AI Provers: An Experience Report*. AIMACS workshop at CAV, 2026. arXiv:2605.30106. https://arxiv.org/abs/2605.30106.

[7] N. Kobeissi. *Verification Theatre: False Assurance in Formally Verified Cryptographic Libraries*. IACR Cryptology ePrint Archive, Report 2026/192, 2026. https://eprint.iacr.org/2026/192.

[8] R. Delmas, Z. Hassan, Q. Hu, R. Kumar, F. R. Monteiro, T. Nguyen, A. Palacios, C. Val, M. Tautschnig, J. Adam, D. Schwartz-Narbonne, and C. Zech. *Kani: A Model Checker for Rust*. In *39th IEEE/ACM Int. Conf. on Automated Software Engineering (ASE), Industry Showcase*, 2026. arXiv:2607.01504. Tool docs: https://model-checking.github.io/kani/.

[9] S. Ho and J. Protzenko. *Aeneas: Rust Verification by Functional Translation*. In *Proc. ACM Program. Lang. (ICFP)*, 2022. (Charon is the companion Rust-to-LLBC front end.) https://github.com/AeneasVerif/aeneas.

[10] Aeneas issue #1221: silent degradation to a bodyless axiom on a `return` nested two loop-levels deep (reported). https://github.com/AeneasVerif/aeneas/issues/1221.

[11] K. Khazem and M. Tautschnig. *CBMC Path: A Symbolic Execution Retrofit of the C Bounded Model Checker (Competition Contribution)*. In *TACAS* (LNCS 11429), Springer, 2019.

[12] J. Katz, Z. Hanna, and N. Dershowitz. *Space-Efficient Bounded Model Checking*. arXiv:0710.4629, 2007, on BMC memory scaling from loop unrolling.